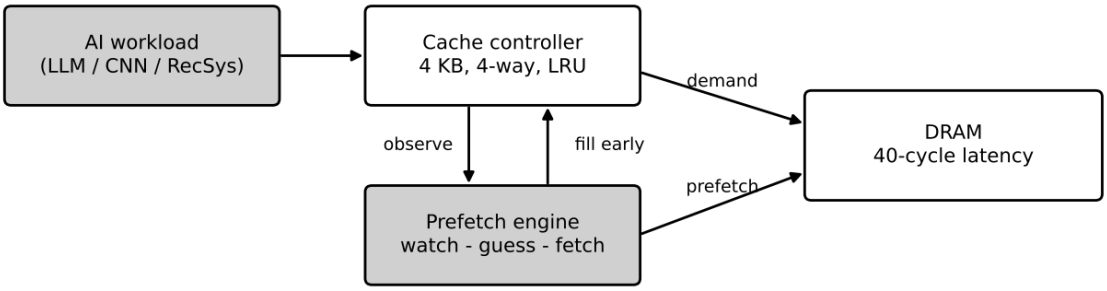
MICRON AI-AWARE CACHE & PREFETCH ENGINE
RTL CHALLENGE

Design, Implementation, Simulation and Performance Evaluation of an Intelligent Cache and Hardware Prefetching Architecture



Fetch data before it is needed: fewer DRAM trips, more cache hits

Event	Micron hackathon - AI-Aware Cache & Prefetch Engine RTL Challenge
Author	Kotapati Kishore (Roll No. EE26E016)
Affiliation	B.Tech EEE, NIT Manipur; final year at IIT Madras (Dept. of Electrical Engineering, student exchange)
Date	1 November 2026
Status	Complete: RTL implemented, verified on two simulators, performance measured
Headline result	2.26x geometric-mean speedup over a cache without prefetching (9 workloads); 2.06x on AI-inspired workloads; 3.03x with 120-cycle DRAM
Source code	https://github.com/Kishore-Kotapati/micron_cache_prefetch

Table of Contents

Executive Summary . . . 7

1. Introduction . . . 8

1.1 Background: AI workloads and the memory wall . . . 8

1.2 Objectives, scope and expected outcomes . . . 8

2. Theoretical Foundations . . . 9

2.1 Memory hierarchy, SRAM and DRAM . . . 9

2.2 Cache lines, tags, index, valid and dirty bits . . . 9

2.3 Locality . . . 9

2.4 Cache organizations . . . 9

2.5 Replacement policies . . . 10

2.6 Access patterns . . . 10

2.7 Prefetching . . . 10

2.8 Measuring a prefetcher . . . 10

2.9 Memory access latency . . . 10

3. System Architecture . . . 11

3.1 Components and data flow . . . 11

3.2 Demand, hit and miss flow . . . 11

3.3 Prefetch request and response flow . . . 12

3.4 Arbitration and correctness rules . . . 12

4. Cache Architecture and Mathematical Design . . . 13

4.1 Parameters (from rtl/cache_controller.v and rtl/top_module.v) . . . 13

4.2 Derivation of the address fields . . . 13

4.3 Storage cost . . . 14

4.4 Why these choices . . . 14

5. Memory Trace Generation and Analysis (Level 1) . . . 14

6. Prefetching Algorithm (Level 3) . . . 16

6.1 Stream tracking without a program counter . . . 16

6.2 Mathematical formulation . . . 16

6.3 Worked example (48-byte stride, degree 2) . . . 16

6.4 Behaviour per access type . . . 17

6.5 Innovation 1 - accuracy-feedback adaptive degree . . . 17

6.6 Innovation 2 - accuracy-gated hybrid kick-off . . . 17

6.7 Innovation 3 - accuracy-adaptive insertion . . . 17

7. RTL Implementation (Level 4) . . . 17

7.1 Cache controller: hit detection and arbitration (excerpt) . . . 18

7.2 Stride predictor: training and trigger (excerpt) . . . 18

7.3 Replacement policy: next-age logic (excerpt) . . . 18
7.4 Testbench architecture . . . 18
8. Tools and Development Environment . . . 19
8.1 Commands . . . 19
8.2 Synthesis check (Yosys) . . . 19
9. Simulation and Verification . . . 20
9.1 Self-checks in every simulation . . . 20
9.2 Regression: 9 traces x 6 architectures . . . 20
9.3 Second simulator . . . 21
9.4 Cocotb directed and random tests . . . 21
9.5 Coverage of required scenarios . . . 21
10. Waveform Analysis . . . 22
11. Performance Evaluation (Level 5) . . . 23
11.1 Worked calculations (LLM-inspired workload) . . . 24
11.2 Comparison of all architectures . . . 26
11.3 Distance to a perfect cache . . . 26
11.4 Sensitivity to DRAM latency . . . 26
11.5 Design-space exploration . . . 27
12. AI Workload Analysis . . . 27
13. Innovation and Optimization . . . 28
14. Challenges, Debugging and Observations . . . 28
15. Limitations and Future Work . . . 29
16. Conclusion . . . 29
References . . . 29
Appendix A - Project File Inventory . . . 30
Appendix B - Full Results Table (all architectures) . . . 30
Appendix C - Verification Checklist . . . 32

List of Figures

Figure 1: Memory hierarchy used in this project 8
Figure 2: Why prefetching helps: the DRAM trip overlaps with computation 8
Figure 3: System block diagram (DRAM model = simulation only) 11
Figure 4: Demand finite-state machine of cache_controller 12
Figure 5: Address decomposition: Address = Tag | Index | Offset 13
Figure 6: Cache organization: 64 sets x 4 ways 14
Figure 7: Address progression of every workload 15
Figure 8: Structure of the prediction engine 16
Figure 9: Yosys drawing of the synthesized top level (cache_controller and prefetch_engine) 20
Figure 10: GTKWave - final design (demo_final.vcd), 0 to 5886 ns 22
Figure 11: GTKWave - baseline without prefetching (demo_baseline.vcd), 0 to 13816 ns 22
Figure 12: Execution-time speedup of every architecture (higher is better) 24
Figure 13: Baseline (A) vs final design (B): miss rate and execution cycles 24
Figure 14: Demand hit rate 25
Figure 15: Measured average memory access latency (lower is better) 25
Figure 16: DRAM traffic relative to the baseline (cost of prefetching) 25
Figure 17: Prefetch accuracy including late prefetches 26
Figure 18: Speedup vs DRAM latency for every workload 27

List of Tables

Table 1: Summary of outcomes 7
Table 2: Project objectives and completion status 8
Table 3: Cache organizations compared 9
Table 4: Prefetch metrics 10
Table 5: System components 11
Table 6: Arbitration rules that guarantee demand correctness 12
Table 7: Implemented cache parameters 13
Table 8: Synthetic workloads 14
Table 9: Trace analysis results (python/analyze_traces.py) 15
Table 10: Step-by-step predictor behaviour 16
Table 11: RTL modules 17
Table 12: Tools and versions 19
Table 13: Yosys generic-gate synthesis results 19
Table 14: Self-checks in tb_system.v 20
Table 15: Golden-model comparison of the baseline cache 20
Table 16: Cocotb tests (results/cocotb_log.txt) 21
Table 17: Scenario coverage 21
Table 18: Signal-by-signal waveform comparison 22
Table 19: Raw counters, baseline (A) vs final design (B) 23
Table 20: Derived metrics, baseline (A) vs final design (B) 23
Table 21: Average speedup of every architecture 26
Table 22: Achieved vs maximum possible speedup 26
Table 23: Speedup vs DRAM latency (python/latency_sweep.py) 26
Table 24: AI-inspired workloads and the predictor's behaviour 27
Table 25: Innovation summary 28
Table 26: Engineering issues encountered and resolved 28
Table 27: Limitations and future work 29
Table 28: Source inventory 30
Table 29: All 54 simulations (results/performance_summary.csv) 30
Table 30: Final acceptance checklist 32

List of Abbreviations

AMAT	Average Memory Access Time
CNN	Convolutional Neural Network
CPU	Central Processing Unit
DRAM	Dynamic Random-Access Memory
FSM	Finite-State Machine

LLM	Large Language Model
LRU	Least Recently Used
MRU	Most Recently Used
MSHR	Miss Status Holding Register
RecSys	Recommendation System
RTL	Register-Transfer Level
SRAM	Static Random-Access Memory
VCD	Value Change Dump (waveform file)
GEMV	General Matrix-Vector multiply
KV	Key/Value (attention cache)

List of Symbols and Notation

A(n)	byte address of the n-th demand access of a stream
S(n)	stride: A(n) - A(n-1), in bytes
D	prefetch degree (number of lines fetched ahead), 1-7, adaptive
C	2-bit confidence counter (0-3)
L	cache line size = 16 B
N_sets, N_ways	number of sets (64) and ways (4)
t_hit, t_miss	hit time (1 cycle) and measured miss latency (about 42 cycles)
T_A, T_B	execution cycles of baseline (A) and final design (B)
x	speedup factor (times faster)

Executive Summary

AI workloads such as large language models, convolutional neural networks and recommendation systems are fast at arithmetic but spend much of their time waiting for data from main memory. Micron's challenge asked for hardware that **watches** the memory addresses a program reads, **guesses** the next ones, and **fetches them early** into the cache. This report documents a complete, working solution built from scratch in one day with only free tools.

The design is a 4 KB, 4-way set-associative cache with true LRU replacement and eight miss-status registers, combined with a multi-stream, confidence-gated stride prefetcher. Three adaptive mechanisms share one small accuracy monitor: (1) the prefetch degree grows or shrinks with measured accuracy, (2) newly started or jumping streams get an immediate next-line kick-off that is switched off when accuracy is low, and (3) prefetched lines are inserted at the LRU position while guesses are poor. Across nine synthetic workloads the final design is **2.26x faster** (geometric mean) than the same cache without prefetching, **2.06x** on the AI-inspired workloads, never slower, and **3.03x** when DRAM latency is raised to 120 cycles.

Table 1: Summary of outcomes

Item	Result
RTL	7 synthesizable Verilog-2005 modules + behavioural DRAM model; Verilator lint: 0 warnings
Functional verification	54/54 simulations pass five self-checks; baseline matches a Python golden model exactly on all 9 traces
Second simulator	All 54 runs give identical counters and cycle counts on Verilator 5.020 and Icarus Verilog 12.0
Cocotb	7/7 directed and constrained-random Python tests pass
Performance	2.26x average, up to 5.2x; regular streams reach 98-99 % of a perfect cache
Hardware cost	Yosys: prefetch engine = 4.9 % of logic cells, 1.8 % of flip-flops
Reproducibility	One command (run_all.sh), reproduced on a second machine (Ubuntu 24.04, WSL); code at https://github.com/Kishore-Kotapati/micron_cache_prefetch

1. Introduction

1.1 Background: AI workloads and the memory wall

A modern processor can complete one or more arithmetic operations every clock cycle, but a read from DRAM takes tens to hundreds of cycles. When a program needs data that is not close to the processor, the processor stalls. A workload is **compute-bound** when its speed is limited by arithmetic, and **memory-bound** when it is limited by how fast data arrives. AI inference is frequently memory-bound: an LLM decoder reads every weight of a layer once per generated token (little reuse), a CNN streams feature maps and channels, and a recommendation model jumps around very large embedding tables. The math units are idle while waiting.

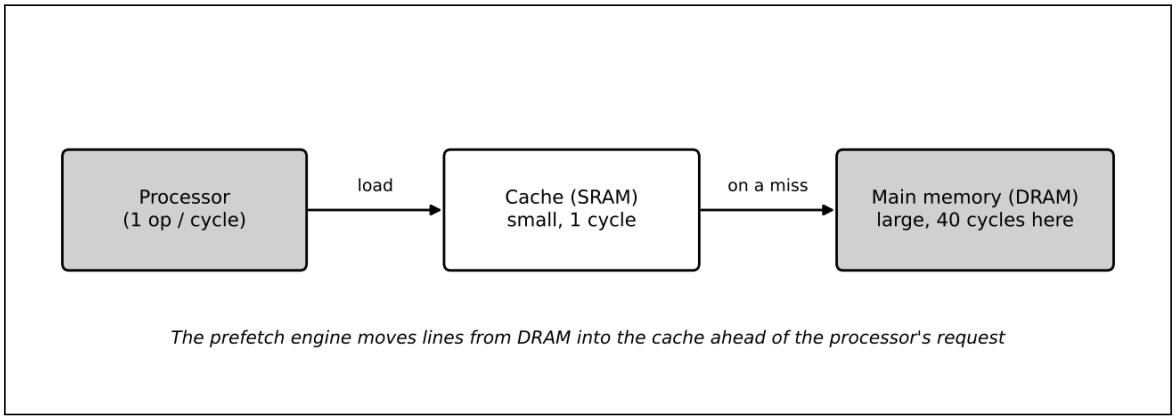


Figure 1: Memory hierarchy used in this project

A cache is a small, fast memory next to the processor that keeps recently used data. It helps when data is reused (temporal locality) or when neighbouring data is used soon (spatial locality). It cannot help with data that has never been touched before. **Prefetching** addresses exactly that case: it predicts future addresses and brings the data in early, overlapping the DRAM delay with useful computation (Figure 2).

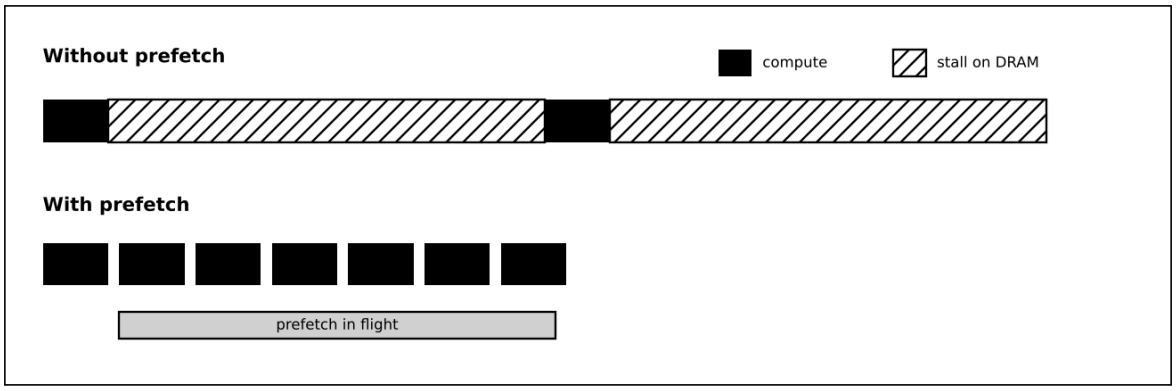


Figure 2: Why prefetching helps: the DRAM trip overlaps with computation

1.2 Objectives, scope and expected outcomes

Table 2: Project objectives and completion status

#	Objective	Status
1	Synthetic memory traces for basic and AI-inspired patterns, with analysis	Done (9 workloads)
2	Baseline cache without prefetching, verified against a golden model	Done (exact match)
3	Hardware stride prefetcher with confidence, duplicate filtering, page protection	Done
4	Synthesizable RTL integrated with a DRAM model and a self-checking testbench	Done
5	Measured comparison of baseline vs prefetching on identical traces	Done (54 runs)

#	Objective	Status
6	Innovation from the slide's four opportunity areas	Done (3 adaptive mechanisms)
7	Optional tools: Verilator, Cocotb, Pandas, Matplotlib	All used

Scope: a single-level, read-only data cache driven by an in-order CPU model that has one outstanding load. Traces are synthetic models of access patterns, not recordings of real AI programs. No claim is made of accelerating actual LLM, CNN or GPU execution.

2. Theoretical Foundations

2.1 Memory hierarchy, SRAM and DRAM

SRAM stores a bit in a cross-coupled latch: fast (about one cycle at cache sizes) but large per bit, so it is used for caches. DRAM stores a bit as charge on a capacitor: dense and cheap but slow, because a row must be opened and sensed, so it is used for main memory. The hierarchy keeps a small amount of fast memory close to the processor and a large amount of slow memory further away. In this project DRAM is modelled with a fixed 40-cycle latency and a bandwidth limit of one new request every 4 cycles.

2.2 Cache lines, tags, index, valid and dirty bits

A cache stores fixed-size **lines** (blocks), here 16 bytes. An address is split into an **offset** (byte within the line), an **index** (which set the line may live in) and a **tag** (which memory block it is). Each stored line has a **valid** bit; on a lookup, the tags of the selected set are compared with the address tag, and an equal tag on a valid line is a **hit**, otherwise a **miss**. A **dirty** bit marks lines modified by stores in a write-back cache. This design is read-only (AI inference reads weights and activations), so no dirty bit is needed; that is listed as a limitation. The design adds a **prefetched** bit per line to measure prefetch usefulness.

2.3 Locality

Temporal locality: an address used now is likely to be used again soon (e.g. the activation vector in GEMV). **Spatial locality**: nearby addresses are likely to be used soon (e.g. consecutive tensor elements). Caches exploit both; prefetchers exploit predictable *future* addresses even without reuse.

2.4 Cache organizations

Table 3: Cache organizations compared

Organization	Placement	Advantages	Disadvantages
Direct-mapped	each line has exactly one slot	cheapest, fastest lookup	conflict misses between streams with equal index
N-way set-associative	a line may use any of N slots in its set	few conflicts, N comparators	slightly more logic, replacement needed
Fully associative	any slot	no conflict misses	one comparator per line (256 here) - expensive

The chosen 4-way set-associative organization removes most conflicts between interleaved AI streams (weights vs activations) while needing only four tag comparators per lookup.

2.5 Replacement policies

When a set is full, one line must be evicted. **FIFO** evicts the oldest inserted line; **Random** evicts any line (cheap, unpredictable); **LRU** evicts the least recently used line and best follows temporal locality. With four ways true LRU is cheap: a 2-bit age per way. This design uses true LRU and adds the option to insert a prefetched line at the LRU position (Chapter 6).

2.6 Access patterns

- **Sequential**: A(n) = A(n-1) + element size, e.g. reading a tensor.
- **Strided**: A(n) = A(n-1) + S with constant S, e.g. one field of a structure array or one channel across pixels.
- **Random**: no relation between consecutive addresses, e.g. hash or embedding lookups.
- **Repeated**: the same set of addresses read again (temporal locality).

2.7 Prefetching

Software prefetching uses instructions inserted by a compiler or programmer; **hardware prefetching** observes the access stream and issues requests itself, without changing the program (this project). A **sequential (next-line)** prefetcher fetches line L+1 after line L; a **stride** prefetcher learns S and fetches A + kS; an **adaptive** prefetcher changes how far ahead and how much it fetches according to measured behaviour.

2.8 Measuring a prefetcher

Table 4: Prefetch metrics

Term	Definition used in this project
Useful prefetch	a demand access hits a line that was brought by a prefetch and not yet used (on time)
Late prefetch	a demand miss finds its line already in flight because of a prefetch; it waits only the remaining time; no second DRAM request
Unused prefetch	issued - useful - late (evicted unused or still resident unused at the end)
Accuracy	useful / issued; also reported: (useful + late) / issued
Coverage	(baseline misses - misses) / baseline misses
Demand access	a load the CPU asked for; prefetches are never counted as accesses

2.9 Memory access latency

$$AMAT = t_{hit} + MissRate \times MissPenalty$$

AMAT is the average number of cycles per load. In this project it is measured directly (sum of all load latencies divided by the number of loads) and also computed with the formula, using the baseline's measured miss penalty.

3. System Architecture

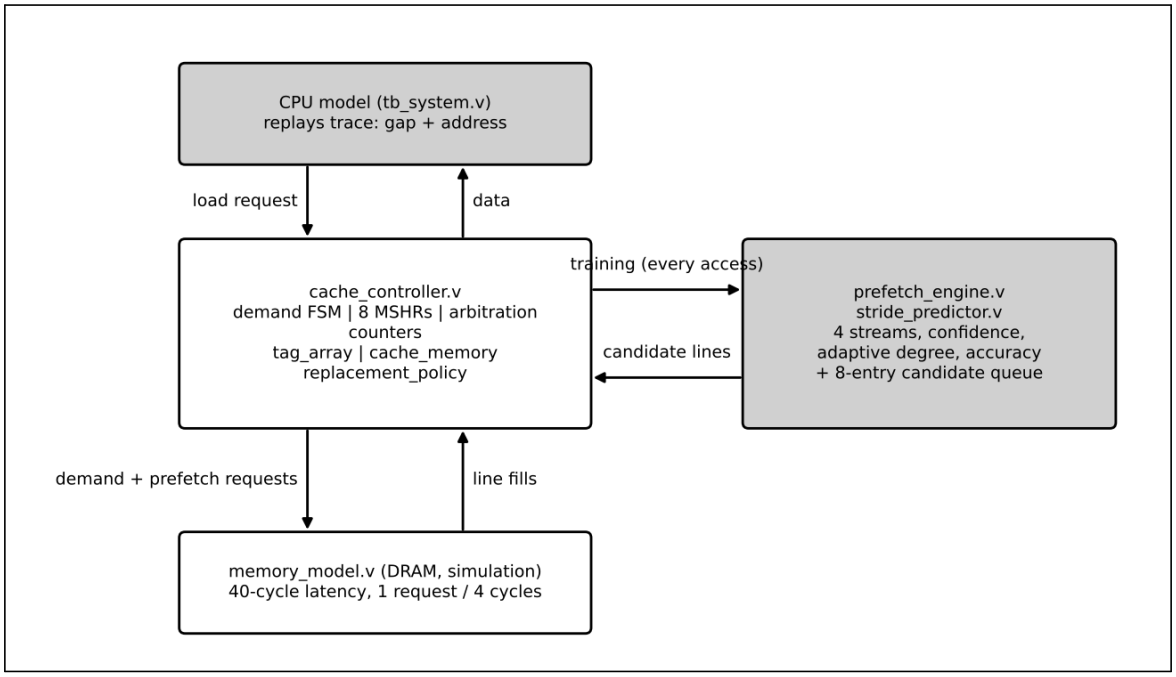


Figure 3: System block diagram (DRAM model = simulation only)

3.1 Components and data flow

Table 5: System components

Block	File	Role
Trace generator	python/generate_traces.py	creates the 9 synthetic workloads (gap + address per line)
CPU model	testbench/tb_system.v	replays a trace: waits the compute gap, issues one load, waits for data, checks it
Cache controller	rtl/cache_controller.v	lookup, hit/miss, demand FSM, 8 MSHRs, prefetch probe, arbitration, counters
Tag array	rtl/tag_array.v	tags, valid and prefetched bits; 3 read ports, 1 install port, 1 clear port
Data array	rtl/cache_memory.v	16-byte line storage, combinational read, write on fill
Replacement	rtl/replacement_policy.v	2-bit LRU age per way, victim selection, MRU or LRU insertion
Stride predictor	rtl/stride_predictor.v	stream table, confidence, triggers, adaptive degree, accuracy flag, hybrid kick-off
Prefetch engine	rtl/prefetch_engine.v	candidate generator (page-bounded), duplicate filter, 8-entry queue
DRAM model	rtl/memory_model.v	40-cycle latency, 1 request per 4 cycles, in-order, 16 deep (simulation only)
Top level	rtl/top_module.v	integration; run-time mode: 0 baseline, 1 next-line, 2 stride, 3 final hybrid

3.2 Demand, hit and miss flow

- **Hit**: the CPU address is split; the four tags of the set are compared; on a match the word is returned the next cycle and the line becomes most recently used. If its prefetched bit was set, the 'useful' counter increments and the bit is cleared.
- **Miss**: if the line is not in flight, the FSM enters S_MISS_REQ, sends the request to DRAM (demand first), allocates an MSHR and waits in S_WAIT. About 40 cycles later the line is installed into the LRU

victim way and the word returned.

- **Merge (late prefetch)**: if an MSHR shows the line is already coming due to a prefetch, no new request is sent; the MSHR is converted to a demand fill and the CPU waits only the remaining time.

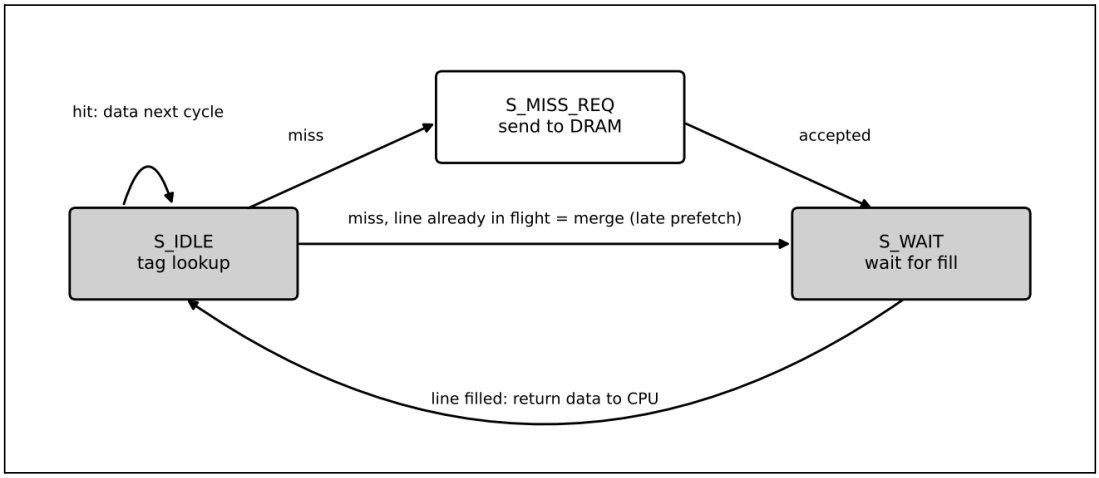


Figure 4: Demand finite-state machine of cache_controller

3.3 Prefetch request and response flow

Every demand lookup is also sent to the predictor (training). When confident, the predictor triggers a set of candidate line addresses; the generator pushes them one per cycle into an 8-entry queue. Each cycle the controller probes the queue head on a dedicated tag port and in the MSHR table: if the line is already cached or in flight it is dropped (filtered). Otherwise it is issued to DRAM when the port is free and at least two MSHRs are free. When the line returns it is installed with its prefetched bit set.

3.4 Arbitration and correctness rules

Table 6: Arbitration rules that guarantee demand correctness

Rule	Implementation	Reason
Demand has priority on the DRAM port	no prefetch issue in a lookup cycle or in S_MISS_REQ	prefetching never delays a demand miss
One array update per cycle	mem_resp_ready = !do_lookup	tag, data and LRU arrays never see two writers
One MSHR reserved for demand	prefetch requires >= 2 free MSHRs	a prefetch burst cannot block the CPU
No duplicates	probe cache + MSHRs before issue; queue de-duplicates	no line stored twice, no wasted DRAM trip
No page crossing	generator stops at 4 KB boundary	never predicts into unrelated memory
Valid/ready handshakes	CPU, DRAM request and response ports	clean, stall-safe interfaces

4. Cache Architecture and Mathematical Design

4.1 Parameters (from rtl/cache_controller.v and rtl/top_module.v)

Table 7: Implemented cache parameters

Parameter	Symbol	Value
Address width	-	32 bits (byte address)
Capacity	C	4096 B (4 KB)
Line size	L	16 B (4 x 32-bit words)
Associativity	N_ways	4
Replacement	-	true LRU, 2-bit age per way; optional LRU insertion of prefetches
MSHRs	-	8 (one reserved for demand)
Hit time	t_hit	1 cycle
DRAM latency / bandwidth	-	40 cycles / 1 request per 4 cycles
Write policy	-	read-only

4.2 Derivation of the address fields

$$Number\ of\ lines = C / L = 4096\ B / 16\ B = 256\ lines$$

$$Number\ of\ sets = lines / N_{ways} = 256 / 4 = 64\ sets$$

$$Offset\ bits = \log_2(L) = \log_2(16) = 4\ bits$$

$$Index\ bits = \log_2(N_{sets}) = \log_2(64) = 6\ bits$$

$$Tag\ bits = 32 - 6 - 4 = 22\ bits$$

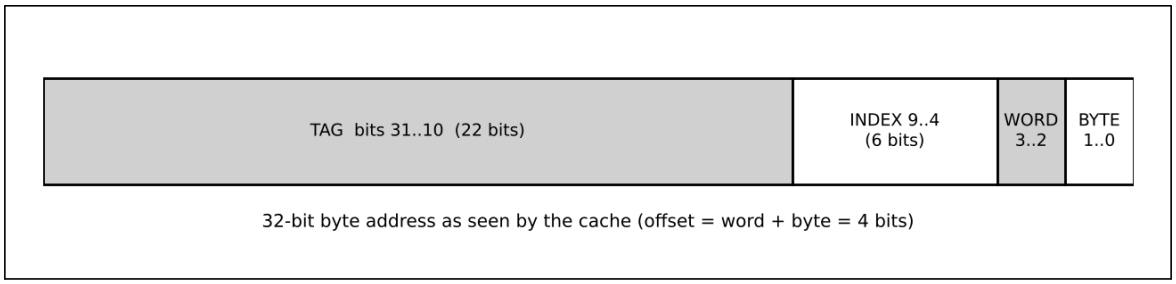


Figure 5: Address decomposition: Address = Tag | Index | Offset

Worked example: address 0x00100A34

Offset = 0x00100A34 mod 16 = 0x4, i.e. word (bits 3..2) = 1. Index = (0x00100A34 / 16) mod 64 = 0x100A3 mod 64 = 35, so the line can only live in set 35. Tag = 0x00100A34 / 1024 = 0x402. The line address used for DRAM requests is 0x00100A34 / 16 = 0x00100A3 (28 bits).

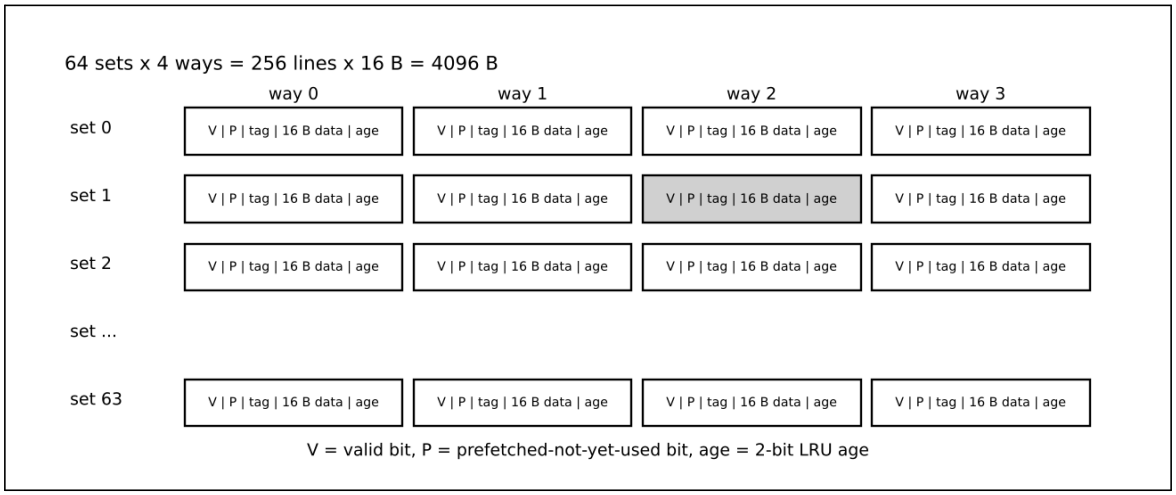


Figure 6: Cache organization: 64 sets x 4 ways

4.3 Storage cost

Per line: 22 tag bits + 1 valid + 1 prefetched + 2 LRU age bits + 128 data bits = 154 bits. For 256 lines: 256 x 154 = 39,424 bits. Yosys synthesis reports 32,768 data flip-flops + 6,144 tag/valid/prefetched flip-flops + 512 LRU flip-flops = 39,424, which matches the calculation exactly (Chapter 8).

4.4 Why these choices

4 KB is deliberately small so that the workloads (8-60 KB footprints, Table 9) do not fit, as real AI models do not fit real caches. 16-byte lines keep the prefetcher's work visible. LRU was chosen because a Python golden model of LRU can verify the baseline exactly. Alternatives were considered analytically (Table 3); the XOR-hashed index was simulated and rejected (Chapter 13).

5. Memory Trace Generation and Analysis (Level 1)

No traces were provided, so python/generate_traces.py creates them. Each line of a trace is '*compute-gap hex byte address*'; the gap is the number of compute cycles before the load. A fixed random seed makes every trace reproducible. All traces are **synthetic models** of access patterns, not recordings of real AI programs.

```
# traces/stride.trace (first lines)      # traces/cnn.trace (first lines)
3 00200000                               4 00a00000
1 00200030                               6 00a00004
2 00200060                               4 00a00008
```

Table 8: Synthetic workloads

Trace	Loads	Model of
sequential	8192	streaming a 32 KB tensor
stride	3000	48-byte stride (3 lines, not line aligned)
random	4000	uniform random words in 1 MB
repeated	3840	1.5 KB working set read 10x in the same shuffled order
mixed	6000	sequential stream + 32 B stride stream + 25 % random, interleaved
capacity	6144	8 KB array (2x cache) read 3 times
cnn	8560	3x3 conv with line-buffer row streams + reused weights; 1x1 conv walking 16 channels (256 B stride)
llm	8320	GEMV: weights streamed once + activation vector reused; then KV-cache scans
recsys	9600	8 Zipf-skewed embedding lookups per sample (64 B rows) + small reused MLP block

Table 9: Trace analysis results (python/analyze_traces.py)

Trace	Footprint (KB)	Temporal reuse %	Adjacent-line %	Stride repeat %	LRU hit %	Top strides
sequential	32.0	0.0	100.0	100.0	75.00	+4B:8191
stride	46.9	0.0	0.0	100.0	0.00	+48B:2999
random	60.6	0.7	0.0	0.0	0.28	+516516B:2
repeated	1.5	90.0	2.1	0.0	97.50	+36B:40
mixed	60.4	0.3	21.3	12.4	34.55	+4B:1275
capacity	8.0	66.7	100.0	99.9	75.00	+4B:6141
cnn	12.3	63.2	72.0	90.3	80.16	+4B:6160
llm	20.3	37.6	23.1	21.5	84.38	+4B:1918
recsys	13.5	64.0	95.3	90.6	88.33	+4B:9150

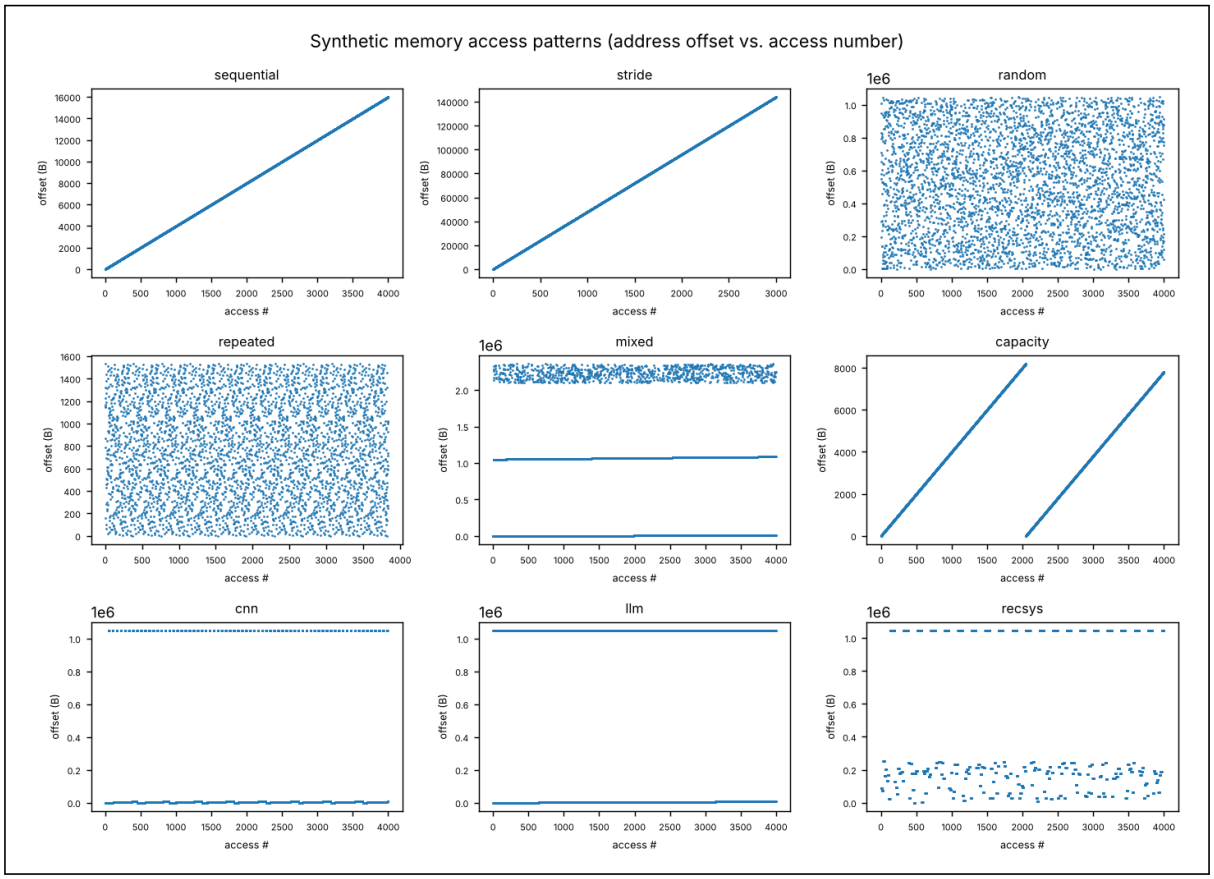


Figure 7: Address progression of every workload

The plots show what each workload offers a prefetcher: straight lines (sequential, stride, capacity) are perfectly predictable; the random cloud is not; mixed and recsys combine predictable streams with unpredictable jumps; CNN and LLM consist of several interleaved regular streams far apart in memory. 'Stride repeat %' (share of steps equal to the previous step) is the quickest indicator of prefetchability.

6. Prefetching Algorithm (Level 3)

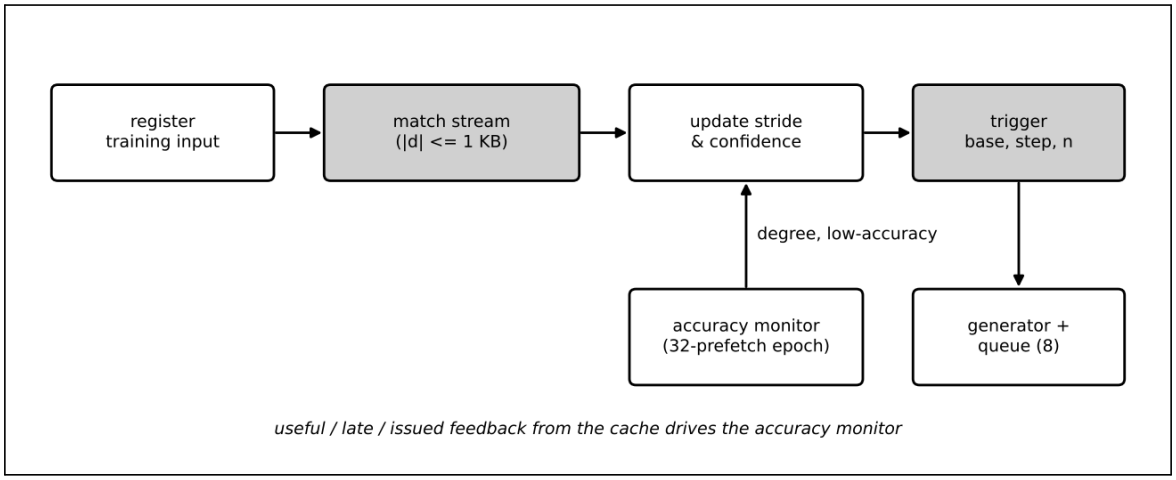


Figure 8: Structure of the prediction engine

6.1 Stream tracking without a program counter

The traces contain only addresses, so streams are separated by address region: an access belongs to the first table entry whose last address is within +/-1 KB. The table has four entries, each holding the last address (32 b), stride (12 b signed, bytes) and a 2-bit confidence C. If no entry matches, a free entry is allocated, else one with C = 0, else round-robin, so random accesses only replace untrained entries.

6.2 Mathematical formulation

$$S(n) = A(n) - A(n-1)$$

$$A_{predicted}(n+1) = A(n) + S(n)$$

$$A_{prefetch,k} = A(n) + k \times S_{eff},\ k = 1 \ldots D$$

These equations are valid when the stride is stable, which the confidence counter checks: if S(n) = S(n-1), C increases (saturating at 3); otherwise C decreases, and when C <= 1 the new difference becomes the stride. Prefetches are triggered only when C >= 2 and the access moved into a new cache line. **Difference between theory and RTL**: strides smaller than a line are rounded up to one line in the stream direction (S_{eff} = +/-16 B), because fetching the same line again is useless; candidates are generated by repeated addition (no multiplier); generation stops at a 4 KB page boundary.

6.3 Worked example (48-byte stride, degree 2)

Table 10: Step-by-step predictor behaviour

Load	Address	S(n)	State after	Action
1	0x2000	-	new entry, S = 0, C = 0	none
2	0x2030	+48	S := 48, C = 0	none
3	0x2060	+48	C = 1	none (not confident)
4	0x2090	+48	C = 2, new line	prefetch 0x20C0 and 0x20F0
5	0x20C0	+48	C = 3	hit on prefetched line; prefetch 0x2120 (0x20F0 filtered as duplicate)

6.4 Behaviour per access type

- **Sequential** (S = 4 B): confident after 3 accesses; triggers once per new line; with adaptive degree it runs 7 lines ahead.
- **Stride** (S = 48 B): tracked exactly in bytes; each access is a new line, each triggers.
- **Random**: differences never repeat, C stays 0, nothing is triggered; the kick-off is gated off by low accuracy.
- **Mixed**: each regular stream keeps its own entry; random accesses only replace C = 0 entries.

6.5 Innovation 1 - accuracy-feedback adaptive degree

Every 32 issued prefetches, the number used (useful + late) is counted. At least 75 % used: degree + 1 (maximum 7). Fewer than 25 %: degree - 1 (minimum 1). Below 50 % a low-accuracy flag is raised for innovations 2 and 3.

6.6 Innovation 2 - accuracy-gated hybrid kick-off

When a new stream starts with a miss, or a known stream breaks its stride on a miss (e.g. a CNN moving to the next feature-map row), two next-line prefetches are issued immediately while the stride re-trains, unless the low-accuracy flag is set. The 'jump' case was added after logging every slow CNN load (Chapter 14).

6.7 Innovation 3 - accuracy-adaptive insertion

While the low-accuracy flag is set, prefetched lines are inserted as the LRU line of their set, so a wrong guess is the first line evicted and cannot push out demand data; otherwise they are inserted as MRU so they survive until used. On the first demand hit the line is promoted to MRU.

7. RTL Implementation (Level 4)

All design modules are written in Verilog-2005 with parameters for every dimension, non-blocking assignments in clocked blocks and combinational logic in always @* blocks. They compile with iverilog -g2005 -Wall and pass Verilator --lint-only -Wall with zero warnings. Complete source files are in the repository; key excerpts follow.

Table 11: RTL modules

Module	Purpose	Main inputs	Main outputs	Key registers
cache_controller.v	Brain of the cache: lookup, demand FSM, MSHRs, probe filter, arbitration, counters	cpu_req_valid/addr, mem_req_ready, mem_resp_valid/id/data, cand_valid/line, pf_insert_lru, pf_low_accuracy	cpu_req_ready, cpu_resp_valid/data, mem_req_valid/line/id, mem_resp_ready, train_*, cand_pop, fb_issued/useful, 10 counters	state[1:0], miss_line_r, miss_word_r, m_v/m_pf/m_wait[7:0], m_line[0:7]
tag_array.v	Tags, valid and prefetched bits with 3 read ports (lookup, probe, fill)	3 indices, install, clear	tags/valid/pf per port	tag_mem[256], valid_bits[256], pf_bits[256]
cache_memory.v	16-byte line storage	read idx/way, write idx/way/data	r_data[127:0]	data_mem[256]
replacement_policy.v	True LRU: victim selection, MRU promotion, LRU insertion	q_idx, q_valid, upd_en/idx/way/lru	victim_way	age[64] (4 x 2 bits)
stride_predictor.v	Stream table, confidence, triggers, adaptive degree, accuracy flag, hybrid kick-off	mode, adaptive_en, base_degree, train_*, fb_issued/useful	trig_valid/base/step/count, degree, low_accuracy	s_last/s_stride/s_conf[4], ep_issued/ep_useful, degree

Module	Purpose	Main inputs	Main outputs	Key registers
prefetch_engine.v	Generator and 8-entry de-duplicating queue	trigger, cand_pop	cand_valid, cand_line	g_addr/g_step/g_left/g_page, q_line[8], q_v
memory_model.v	DRAM model (simulation only)	req_valid/line/id, resp_ready	req_ready, resp_valid/id/data	queue of {id, line, due-cycle}, busy counter
top_module.v	Integration and run-time configuration	clk, rst, pf_mode, pf_adaptive, pf_degree, pf_insert_lru, CPU port	CPU response	-

7.1 Cache controller: hit detection and arbitration (excerpt)

```
wire do_lookup = cpu_req_valid && cpu_req_ready;          // S_IDLE only
always @* begin                                          // 4-way tag compare
    hit = 1'b0; hit_way = 0;
    for (i = 0; i < WAYS; i = i + 1)
        if (!hit && a_valid[i] && (a_tags[i*TAG_W +: TAG_W] == rq_tag)) begin
            hit = 1'b1; hit_way = i; end
end
assign mem_resp_ready = !do_lookup;                      // demand lookup first
wire pf_drop  = cand_valid && (pb_cache || pb_mshr);     // already cached / in flight
wire pf_req   = cand_valid && !pf_drop && !do_lookup && (state != S_MISS_REQ) && (free_cnt >= 2);
assign mem_req_valid = (state == S_MISS_REQ) ? dem_req : pf_req;
```

7.2 Stride predictor: training and trigger (excerpt)

```
if (match && mdiff != 0) begin
    s_last[midx] <= tr_addr;
    if (m_same) begin                                     // same stride again
        s_conf[midx] <= m_conf_inc;
        if (m_conf_inc >= CONF_TH && m_line_chg) begin    // confident + new line
            trig_valid <= 1; trig_base <= tr_addr; trig_step <= eff_step; trig_count <= degree;
        end
    end else begin                                        // stride broken
        if (m_conf != 0)  s_conf[midx] <= m_conf - 1;
        if (m_conf <= 1)  s_stride[midx] <= mdiff[STR_W-1:0];
        if (mode == MODE_HYBRID && tr_miss && !low_accuracy)  // re-sync kick-off
            begin trig_valid <= 1; trig_step <= LINE_BYTES; trig_count <= NEW_CNT; end
    end
end
```

7.3 Replacement policy: next-age logic (excerpt)

```
if (i == upd_way)               nxt = upd_lru ? AGE_MAX : 0;   // insert LRU or promote MRU
else if (!upd_lru && a < ref)   nxt = a + 1;                   // others age by one
else if ( upd_lru && a > ref)   nxt = a - 1;
else                            nxt = a;
```

7.4 Testbench architecture

tb_system.v loads a trace with $fscanf, selects the architecture with plusargs (+mode, +adaptive, +degree, +inslru), drives the CPU port after each rising edge, measures every load's latency, compares every returned word with the DRAM content function (addr XOR 0x5A5A5A5A) x 0x9E3779B1, drains outstanding prefetches, and finally runs structural checks on the internal arrays. It prints one RESULT line and TEST PASSED or TEST FAILED.

8. Tools and Development Environment

Table 12: Tools and versions

Tool	Version (author / second machine)	Role in this project
Python 3	3.13 / 3.12.3	trace generation, analysis, golden model, regression, metrics
Icarus Verilog	12.0 / 12.0	main simulator (54 runs, Cocotb tests)
GTKWave	3.3.116 (second machine)	waveform inspection and demo screenshots
Git / GitHub	2.43	version history; repository https://github.com/Kishore-Kotapati/micron_cache_prefetch
Verilator	5.020 / 5.020	lint (0 warnings) and second simulator (--binary --timing)
Cocotb	2.1.0 / 2.1.0	7 Python directed and random tests
Pandas	3.0 / 3.1	result tables
Matplotlib	3.11	all plots
Yosys (extra)	0.33	synthesis to generic gates, block diagram

8.1 Commands

```
bash run_all.sh                       # everything below in ~1 minute
python3 python/generate_traces.py     # Level 1 traces
iverilog -g2005 -Wall -o simulation/build/sim.vvp testbench/tb_system.v rtl/*.v
vvp -n simulation/build/sim.vvp +trace=traces/llm.trace +mode=0                      # baseline
vvp -n simulation/build/sim.vvp +trace=traces/llm.trace +mode=3 +adaptive=1 +degree=2 +inslru=1
python3 python/run_all.py             # 54 simulations + golden model + metrics
python3 python/verilator_crosscheck.py
python3 python/run_cocotb.py
python3 python/latency_sweep.py ; python3 python/design_sweep.py
gtkwave simulation/waveforms/demo_final.vcd simulation/waveforms/demo.gtkw
cd synth && bash run_synth.sh
```

8.2 Synthesis check (Yosys)

Table 13: Yosys generic-gate synthesis results

Block	Logic cells	Flip-flops
cache_memory (data)	65,968	32,768
tag_array	22,953	6,144
cache_controller	3,872	634
replacement_policy	1,855	512
stride_predictor	2,825	309
prefetch_engine (queue)	2,002	424
Total	**99,471**	**40,791**

The whole prefetch engine (predictor + queue) is 4,827 cells (4.9 %) and 733 flip-flops (1.8 %); Yosys check -assert passes. These are generic gates: no standard-cell area, timing, power or placement is claimed.

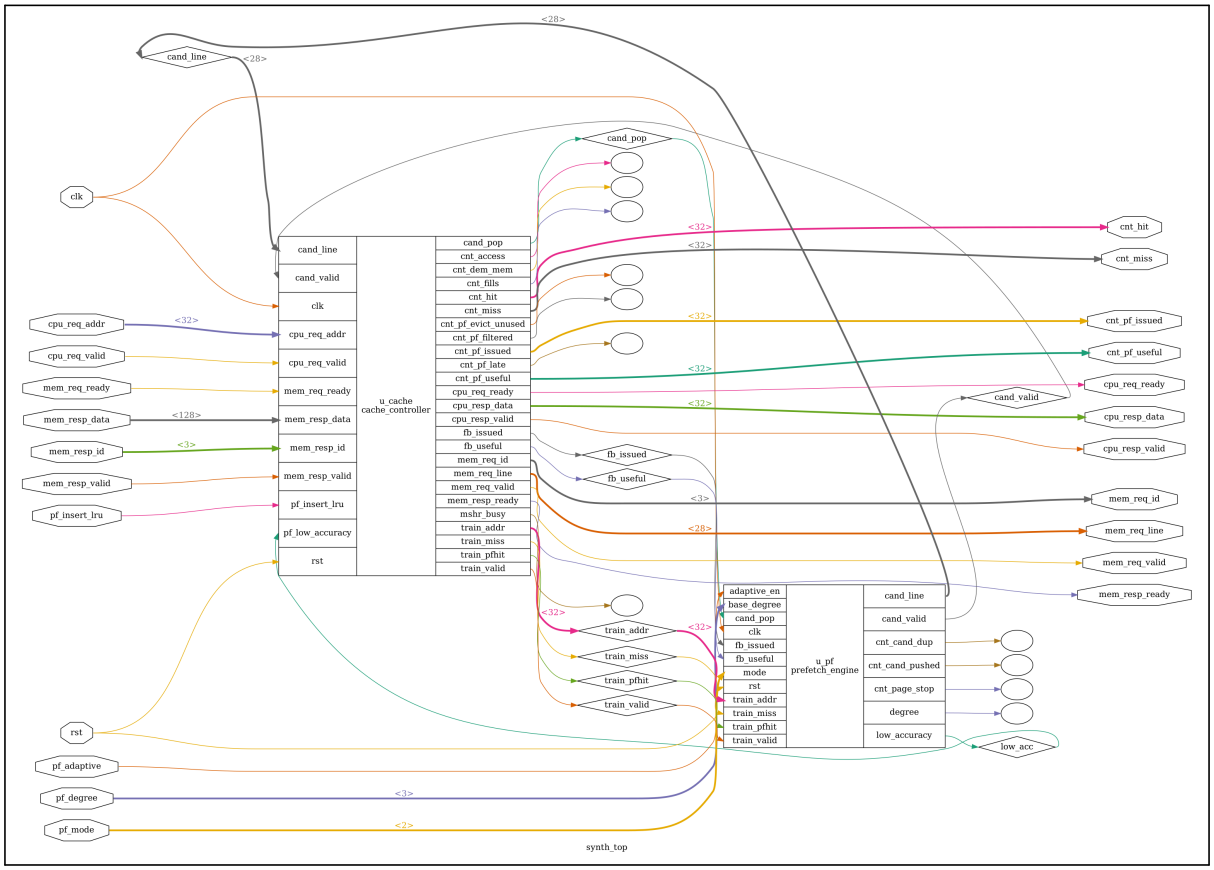


Figure 9: Yosys drawing of the synthesized top level (cache_controller and prefetch_engine)

9. Simulation and Verification

9.1 Self-checks in every simulation

Table 14: Self-checks in tb_system.v

#	Check	Pass criterion
1	Data correctness	every returned word equals the DRAM content function of its address
2	Counter consistency	hits + misses = accesses = trace length
3	No duplicate lines	no tag appears twice among valid ways of any set
4	Prefetch accounting	issued = useful + late + evicted-unused + resident-unused
5	Drain	all MSHRs empty at the end (no lost responses)

9.2 Regression: 9 traces x 6 architectures

The same RTL is run with six configurations: baseline, next-line, stride (fixed degree 2), stride + adaptive degree, + hybrid kick-off, and the final design (+ adaptive insertion). **Result: 54/54 PASS.** The baseline was additionally compared with python/ref_cache_model.py, an independent LRU model; because baseline hits do not depend on timing, they must match exactly.

Table 15: Golden-model comparison of the baseline cache

Trace	Python hits	Python misses	RTL hits	RTL misses	Result
sequential	6144	2048	6144	2048	MATCH
stride	0	3000	0	3000	MATCH
random	11	3989	11	3989	MATCH
repeated	3744	96	3744	96	MATCH

Trace	Python hits	Python misses	RTL hits	RTL misses	Result
mixed	2073	3927	2073	3927	MATCH
capacity	4608	1536	4608	1536	MATCH
cnn	6862	1698	6862	1698	MATCH
llm	7020	1300	7020	1300	MATCH
recsys	8480	1120	8480	1120	MATCH

9.3 Second simulator

The identical testbench and RTL were compiled with Verilator 5.020 (--binary --timing). All 54 runs produced identical counters, cycle counts and self-check results (results/verilator_crosscheck.txt). Output on the second machine:

```
== Verilator cross-check (same testbench, second simulator)
54/54 runs: Verilator results identical to Icarus Verilog (all counters, cycles and self-checks)
```

9.4 Cocotb directed and random tests

Table 16: Cocotb tests (results/cocotb_log.txt)

Test	Objective / stimulus	Expected	Observed	Status
hit_and_miss_latency	cold load then 3 loads in same line	miss ~ DRAM latency; hits 1 cycle	miss 42, hit 1	PASS
lru_replacement	5 lines in one set, re-touch line 0	line 1 evicted, line 0 kept	as expected	PASS
random_vs_golden_model	1500 constrained-random loads	hits/misses = Python model	554 / 946, exact	PASS
prefetch_removes_misses	128-line stream, final design	demand DRAM requests << 128	1 request, 22 misses (21 late)	PASS
demand_during_prefetch	load a line while its prefetch is in flight; unrelated miss with prefetches busy	merge, no 2nd request; miss not delayed	merge 11 cycles; miss 42 with MSHRs 0x1c busy	PASS
no_prefetch_across_page	stream ending at a 4 KB boundary	0 prefetches into next page	30 issued, 0 across	PASS
random_traffic_is_throttled	600 random loads	few prefetches	32 prefetches	PASS

```
** TESTS=7 PASS=7 FAIL=0 SKIP=0   (second machine, Ubuntu 24.04, cocotb 2.1.0)
```

9.5 Coverage of required scenarios

Table 17: Scenario coverage

Required scenario	Covered by
Sequential / stride / random / repeated / mixed	traces of the same names
Cold cache	every run starts from reset
Capacity pressure	capacity trace (8 KB on 4 KB cache)
Prefetch usefulness and accuracy	RTL counters on every run
Demand requests during ongoing prefetches	late merges in every prefetch run; Cocotb test 5
Duplicate prefetch handling	probe filter counters; self-check 3 (no duplicate lines)
Boundary conditions	page-boundary Cocotb test; drain check

10. Waveform Analysis

Waveforms come from a short demonstration trace (96 sequential loads followed by 2 random loads) simulated once with the baseline and once with the final design. The screenshots below were taken in GTKWave on the second machine.

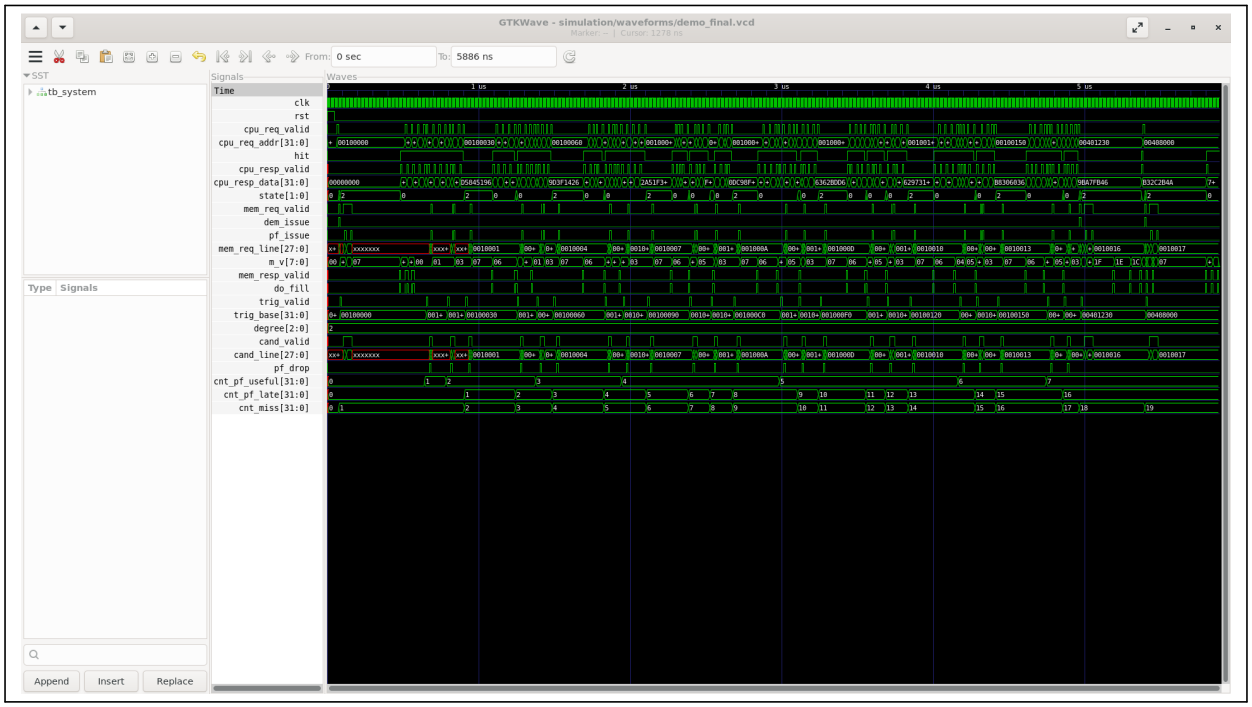


Figure 10: GTKWave - final design (demo_final.vcd), 0 to 5886 ns

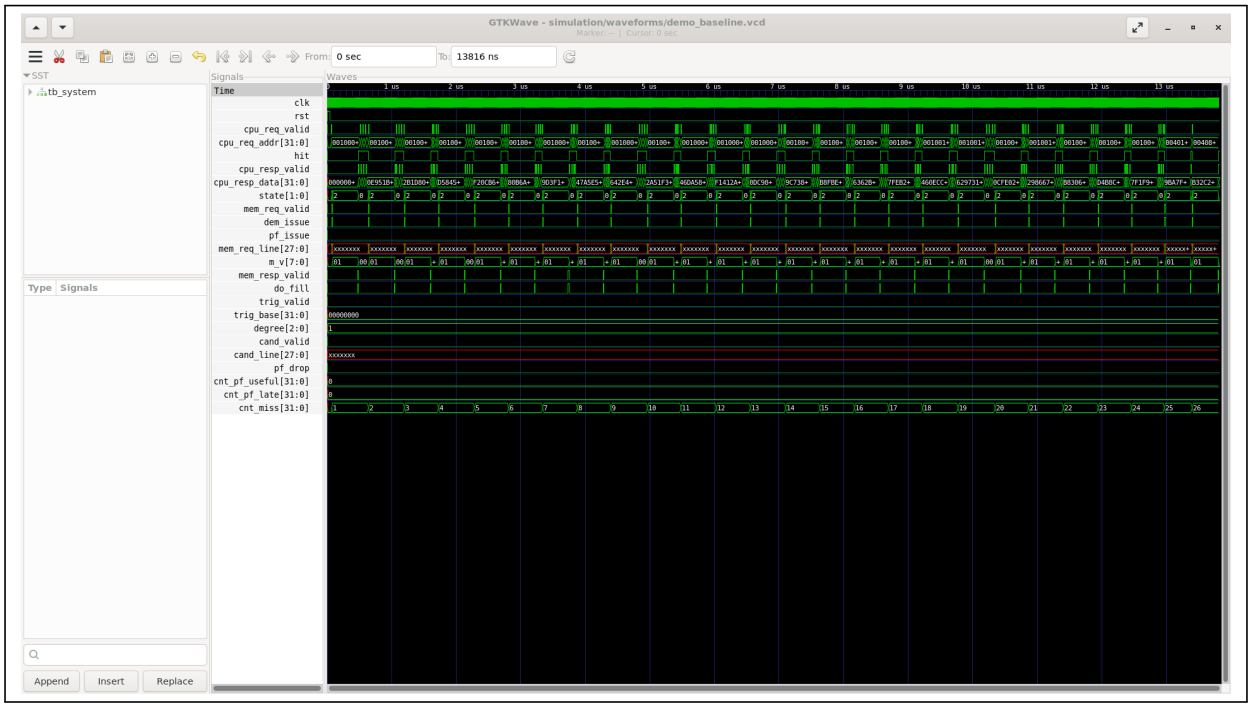


Figure 11: GTKWave - baseline without prefetching (demo_baseline.vcd), 0 to 13816 ns

Table 18: Signal-by-signal waveform comparison

Signal	Baseline	Final design	Meaning
end time	13,816 ns (about 1,382 cycles)	5,886 ns (about 589 cycles)	2.35x faster on the same 98 loads

Signal	Baseline	Final design	Meaning
dem_issue	26 pulses	3 pulses	demand DRAM requests (first load + 2 random loads)
pf_issue	never	many, between CPU loads	prefetches never collide with demand lookups
m_v[7:0]	01 / 00	07, 1F, 1E, 1C	up to 5 requests in flight hide DRAM latency
state[1:0]	2 after every new line	mostly 0	CPU waits far less
trig_base	00000000	00100030, 00100060 ...	predictor triggers ahead of the CPU
cnt_miss / useful / late	26 / 0 / 0	19 / 7 / 16	matches simulation counters
cpu_resp_data	D5845196, 9D3F1426 ...	identical values	same correct data, delivered earlier

Timing sequence (final design). After reset the first loads miss (dem_issue). Once the stream entry reaches confidence 2, trig_valid pulses with trig_base equal to the current address; cand_valid rises and cand_line shows the next lines; pf_issue fires in cycles without a demand lookup; m_v shows several MSHRs busy; mem_resp_valid / do_fill install the lines; later CPU loads hit (hit high, response next cycle), and loads that find their line still in flight increment cnt_pf_late instead of issuing a second request. The two random loads at the end cause the last two dem_issue pulses.

Observation. In the baseline, cand_line and parts of mem_req_line show 'x' (red). This is expected: with prefetching disabled, the candidate queue memory is never written; the value is only used when its valid signal is 1, which never happens in baseline mode. All data checks pass.

11. Performance Evaluation (Level 5)

Architecture A is the baseline cache; architecture B is the final design. Both use identical RTL, traces and DRAM timing; only the run-time prefetch mode differs. Every value below is read from results/performance_summary.csv, which is produced from the simulation logs.

Table 19: Raw counters, baseline (A) vs final design (B)

Trace	Loads	Hits A/B	Misses A/B	Hit % A/B	DRAM A/B	PF issued	Useful	Late
sequential	8192	6144/8145	2048/47	75.0/99.4	2048/2048	2040	2001	39
stride	3000	0/633	3000/2367	0.0/21.1	3000/3011	2972	633	2328
random	4000	11/11	3989/3989	0.3/0.3	3989/4021	32	0	0
repeated	3840	3744/3774	96/66	97.5/98.3	96/98	32	30	0
mixed	6000	2073/4368	3927/1632	34.5/72.8	3927/5652	4108	2295	88
capacity	6144	4608/6091	1536/53	75.0/99.1	1536/1536	1530	1483	47
cnn	8560	6862/8196	1698/364	80.2/95.8	1698/2465	2299	1347	198
llm	8320	7020/8279	1300/41	84.4/99.5	1300/1339	1330	1259	32
recsys	9600	8480/9010	1120/590	88.3/93.8	1120/2001	1549	584	138

Table 20: Derived metrics, baseline (A) vs final design (B)

Trace	AMAT A	AMAT B	Cycles A	Cycles B	Speedup	Impr. %	Accuracy %	Acc.+late %	Coverage %
sequential	11.25	1.07	112571	29172	3.86	74.1	98.1	100.0	97.7
stride	42.00	6.04	133557	25688	5.20	80.8	21.3	99.6	21.1
random	41.89	41.89	177688	177688	1.00	0.0	0.0	0.0	0.0
repeated	2.02	1.71	17357	16127	1.08	7.1	93.8	93.8	31.2
mixed	27.83	11.89	182100	86455	2.11	52.5	55.9	58.0	58.4
capacity	11.25	1.09	84464	22016	3.84	73.9	96.9	100.0	96.5
cnn	9.13	2.02	112549	51680	2.18	54.1	58.6	67.2	78.6
llm	7.41	1.10	82323	29855	2.76	63.7	94.7	97.1	96.8

Trace	AMAT A	AMAT B	Cycles A	Cycles B	Speedup	Impr. %	Accuracy %	Acc.+late %	Coverage %
recsys	5.78	3.17	79479	54379	1.46	31.6	37.7	46.6	47.3
geo-mean					**2.26**				

11.1 Worked calculations (LLM-inspired workload)

$$Hit\ rate_B = 8279 / 8320 = 0.9951 = 99.51\ \%$$

$$Miss\ rate_A = 1300 / 8320 = 15.62\ \%;\ Miss\ rate_B = 41 / 8320 = 0.49\ \%$$

$$Accuracy = useful / issued = 1259 / 1330 = 94.7\ \%$$

$$Coverage = (1300 - 41) / 1300 = 96.8\ \%$$

$$Speedup = T_A / T_B = 82323 / 29855 = 2.757x$$

$$Improvement = (82323 - 29855) / 82323 \times 100 = 63.73\ \%$$

AMAT check (sequential, baseline). Miss rate = 2048 / 8192 = 0.25 and the measured miss latency is 42 cycles, so miss penalty = 42 - 1 = 41 cycles and AMAT = 1 + 0.25 x 41 = 11.25 cycles, identical to the measured 11.25. For the prefetching design the simple formula over-estimates AMAT (1 + 0.0057 x 41 = 1.23 vs measured 1.07) because late prefetches count as misses but wait only part of the latency; the measured value is therefore reported as the primary metric.

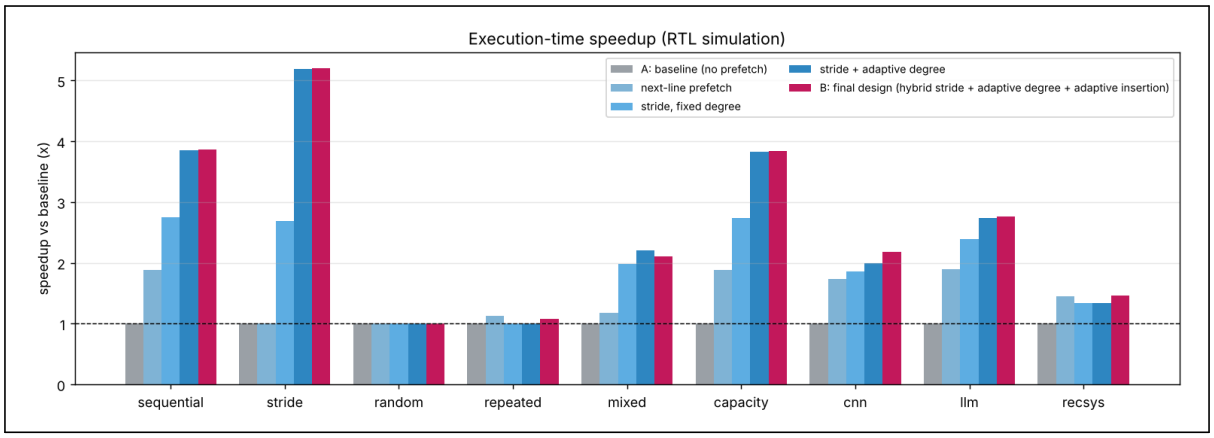


Figure 12: Execution-time speedup of every architecture (higher is better)

X-axis: workload; Y-axis: baseline cycles divided by the architecture's cycles; the dashed line is the baseline. Each bar group adds one idea from left to right, so the contribution of each idea is visible. Regular streams benefit most; random cannot benefit; repeated already fits in the cache.

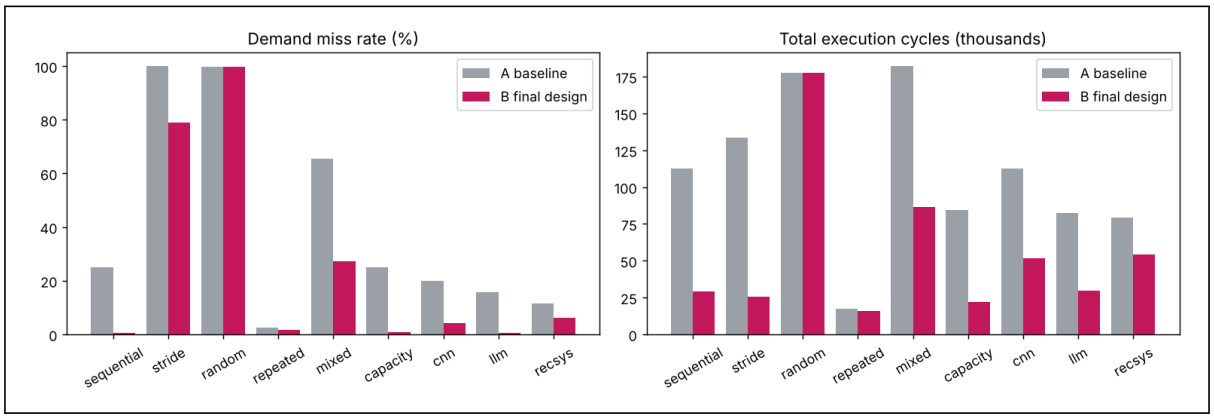


Figure 13: Baseline (A) vs final design (B): miss rate and execution cycles

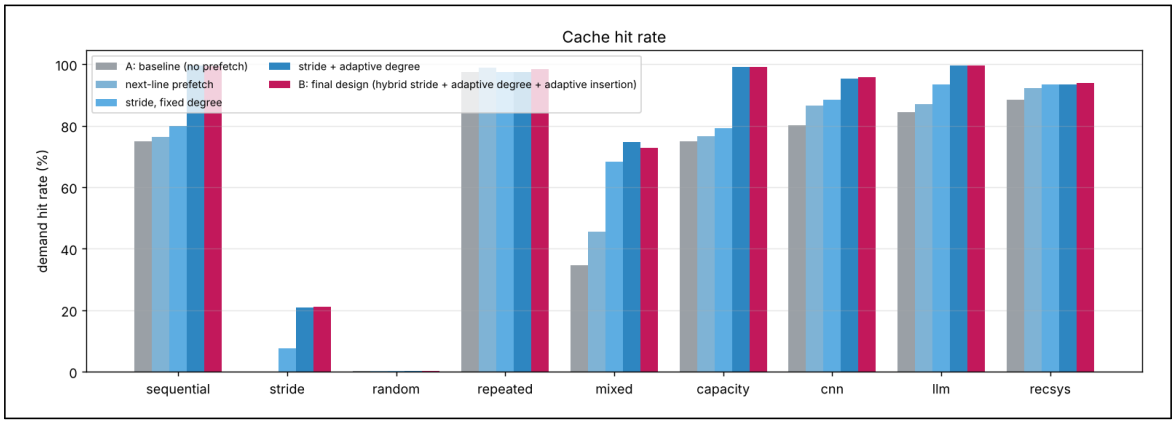


Figure 14: Demand hit rate

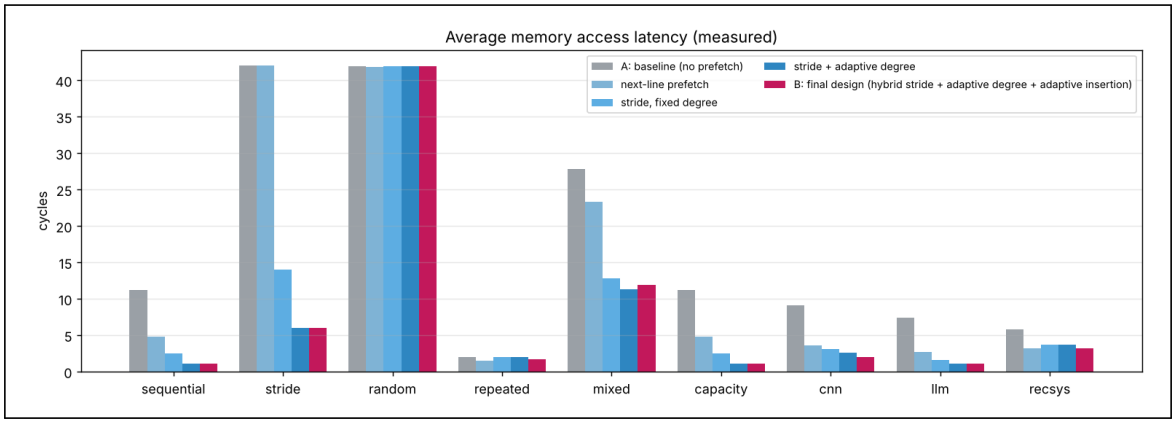


Figure 15: Measured average memory access latency (lower is better)

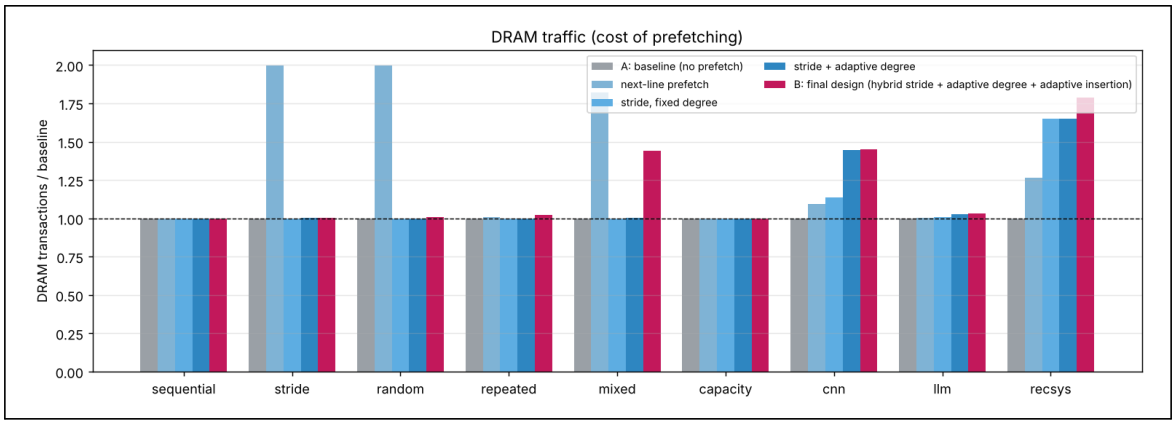


Figure 16: DRAM traffic relative to the baseline (cost of prefetching)

Prefetching is not free: CNN, recsys and mixed use 1.4-1.8x the baseline DRAM traffic; regular streams use the same traffic because the same lines are simply fetched earlier. The next-line prefetcher doubles traffic on random and stride workloads with no benefit, which the accuracy gate avoids.

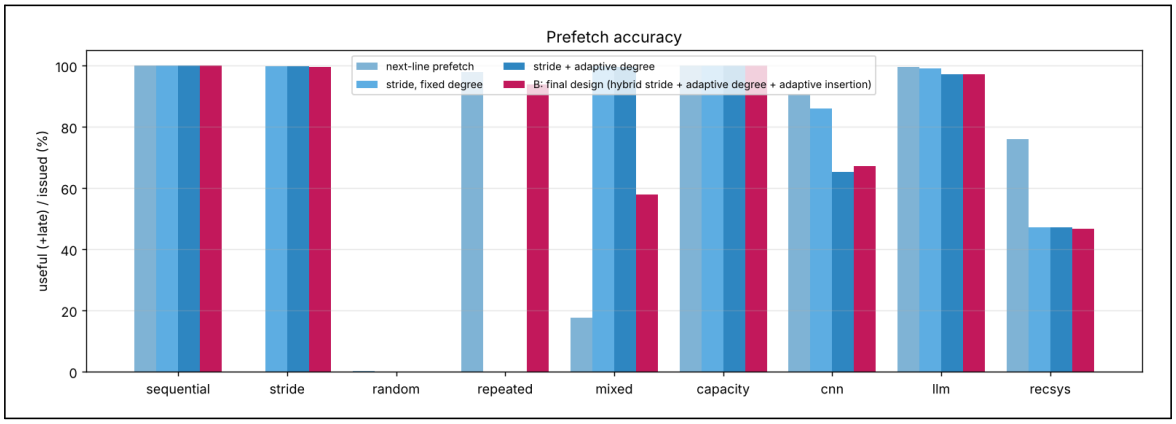


Figure 17: Prefetch accuracy including late prefetches

11.2 Comparison of all architectures

Table 21: Average speedup of every architecture

Architecture	Geo-mean speedup (9 traces)	AI-inspired (cnn, llm, recsys)
A: baseline	1.00x	1.00x
next-line	1.41x	1.68x
stride (fixed)	1.84x	1.81x
stride + adaptive degree	2.21x	1.94x
+ hybrid kick-off	2.25x	2.02x
B: final design	2.26x	2.06x

11.3 Distance to a perfect cache

Because the CPU model is identical, a perfect cache (every load 1 cycle) would take T_A - (sum of latencies) + number of loads cycles. The share of the possible gain achieved is $(T_A - T_B) / (T_A - T_{ideal})$.

Table 22: Achieved vs maximum possible speedup

Trace	Speedup now	Maximum possible	Share of possible gain
sequential	3.86x	3.94x	99.3 %
stride	5.20x	12.65x	87.7 %
random	1.00x	12.57x	0.0 %
repeated	1.08x	1.29x	31.2 %
mixed	2.11x	8.63x	59.4 %
capacity	3.84x	3.93x	99.2 %
cnn	2.18x	2.62x	87.4 %
llm	2.76x	2.84x	98.4 %
recsys	1.46x	2.37x	54.7 %

11.4 Sensitivity to DRAM latency

Table 23: Speedup vs DRAM latency (python/latency_sweep.py)

DRAM latency (cycles)	Geo-mean speedup, 9 traces	AI-inspired
20	1.82x	1.62x
40	2.26x	2.06x
80	2.82x	2.65x
120	3.03x	2.97x

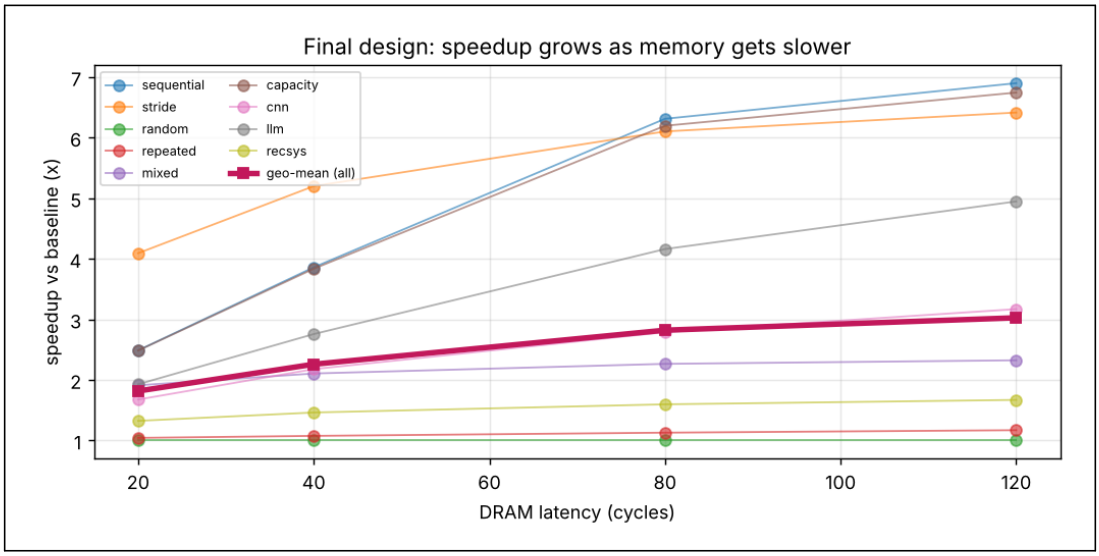


Figure 18: Speedup vs DRAM latency for every workload

X-axis: DRAM latency in cycles; Y-axis: speedup of the final design over the baseline at that latency; one line per workload, thick line = geometric mean. The slower the memory, the more each prefetch hides, so the benefit grows. The stride line flattens above 80 cycles because DRAM bandwidth (one request per 4 cycles) and the 7 prefetch MSHRs become the limit; random stays at 1.0x.

11.5 Design-space exploration

Sixteen combinations of stream-table size (4/8), confidence threshold (1/2), kick-off lines (2/3) and accuracy epoch (16/32) were simulated on all nine traces (results/design_sweep.csv). All lie between 2.22x and 2.24x: the design is robust to its settings. The best combination is less than 1 % better and was tuned on the same traces it would be reported on, so the default settings were kept to avoid over-fitting.

12. AI Workload Analysis

Table 24: AI-inspired workloads and the predictor's behaviour

Workload	Characteristic	How the design handles it	Speedup B (vs next-line)
CNN	row streams per channel, reused weights, 256 B channel stride	stride tracking per region, degree up to 7, re-sync kick-off on every new row	2.18x (1.73x)
LLM	weights streamed once, activation vector reused, KV-cache scans	long regular streams; adaptive degree reaches 7; 97 % of misses removed	2.76x (1.89x)
RecSys	Zipf-skewed random rows of 4 lines + small reused MLP	hybrid kick-off for short new streams; LRU insertion when accuracy is low	1.46x (1.45x)

Where it struggles: the first line of a randomly chosen embedding row cannot be predicted by any address-based prefetcher, which keeps recsys at 55 % of the possible gain; CNN pointwise channels exactly 256 B apart map to the same four cache sets (conflict misses). These are synthetic models; no claim is made about real model execution.

13. Innovation and Optimization

Table 25: Innovation summary

Opportunity (official slide)	What was done	Status	Measured effect
Intelligent Prefetching	accuracy-feedback adaptive degree 1-7	implemented	geo-mean 1.84x -> 2.21x
Novel Prediction Algorithms	accuracy-gated hybrid kick-off for new and jumping streams	implemented	CNN 2.00x -> 2.18x; repeated 1.00x -> 1.08x
Adaptive Cache Policies	accuracy-adaptive LRU/MRU insertion of prefetched lines	implemented	recsys 1.37x -> 1.46x, 9 % less DRAM traffic; mixed 2.19x -> 2.11x
AI-Aware Optimization	behaviour adapts to traffic type (long streams vs short bursts vs random)	implemented via the above	AI geo-mean 2.06x
XOR-hashed set index	simulated for baseline and final design	evaluated, not adopted	CNN +3.4 %, recsys -1.3 %
Parameter tuning	16-point design sweep	evaluated, defaults kept	all within 2.25-2.27x

All three adaptive mechanisms share one accuracy monitor (two 6-bit counters), so the 'intelligence' costs very little hardware (4.9 % of cells for the whole prefetch engine).

14. Challenges, Debugging and Observations

Table 26: Engineering issues encountered and resolved

Problem	Evidence	Cause	Fix / outcome
Always inserting prefetches at LRU hurt multi-stream traffic	first full regression: mixed and CNN slower than without the policy	prefetched lines were evicted by other streams' fills before being used	made insertion adaptive (LRU only while accuracy < 50 %)
Throttling too aggressive on recsys	adaptive degree dropped to 1 and recsys became slower than fixed degree 2	50 % lower threshold too high for short streams	lower threshold changed to 25 %
Prefetches late on bandwidth-bound stride traffic	21 % timely accuracy at degree 4	not far enough ahead	maximum degree raised to 7 (3-bit field); stride 4.13x -> 5.20x with the same 8 MSHRs
Recsys behind next-line	1.37x vs 1.45x	4-line rows end before the stride trains	hybrid kick-off for new streams (1.46x)
CNN first lines of every row missed	per-load miss log (+misslog) of the CNN trace	row jump inside the 1 KB window looked like a broken stride, so no kick-off	re-sync kick-off on stride breaks; CNN 2.02x -> 2.18x
Verilator width warnings	lint report	implicit width extension in index arithmetic	concatenated {index, way} addressing; lint clean
Two Cocotb tests failed on first run	assertion messages	the tests were wrong (late merges counted as full misses; a merge test loaded an already-arrived line)	tests corrected; RTL unchanged
Second machine: Verilator build failed	'Cannot write simulation/verilator/obj/...'	build folder excluded from the archive; Verilator does not create parent folders	script creates the folder; 54/54 identical
Second machine: tool versions	devel OSS CAD Suite builds and missing pip packages	Ubuntu 24.04 packages match the reference versions	apt packages + Python venv for Cocotb

15. Limitations and Future Work

Table 27: Limitations and future work

Limitation (actual)	Possible future work
synthetic traces, not real AI programs	replay traces captured from real model runs
read-only cache	write-back with dirty bits
single level, CPU with one outstanding load	multi-level hierarchy, out-of-order core model
streams matched by address region (no PC)	PC-indexed stride table
fixed-latency DRAM	banks, rows, refresh
global accuracy for all streams	per-stream accuracy and degree
conflict misses on power-of-two strides	set-index hashing (evaluated)
generic-gate synthesis only; flip-flop arrays	SRAM macros, OpenROAD flow for area, timing, power and placement

16. Conclusion

Starting from four presentation slides and no starter code, a complete AI-aware cache and prefetch engine was designed, implemented in synthesizable Verilog, and verified with five independent methods: in-simulation self-checks, an exact Python golden model, a second simulator with bit-identical results, seven Cocotb tests and lint/synthesis checks. The multi-stream stride prefetcher with three accuracy-driven adaptive mechanisms makes the same 4 KB cache **2.26x faster** on average across nine workloads and **2.06x** on AI-inspired ones, never slower, and **3.03x** with slower 120-cycle memory. On regular streams it reaches 98-99 % of a perfect cache, at a hardware cost below 5 % of the design. The remaining gap is dominated by unpredictable (random) accesses and DRAM bandwidth, and the main limitations - synthetic traces, a read-only single-level cache and generic-gate synthesis - are stated openly. All results are reproducible with one command from https://github.com/Kishore-Kotapati/micron_cache_prefetch.

References

[1] J. L. Hennessy and D. A. Patterson, *Computer Architecture: A Quantitative Approach*, 6th ed. Morgan Kaufmann, 2017.

[2] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design: The Hardware/Software Interface*, Morgan Kaufmann.

[3] N. P. Jouppi, "Improving direct-mapped cache performance by the addition of a small fully-associative cache and prefetch buffers," in *Proc. ISCA*, 1990.

[4] J.-L. Baer and T.-F. Chen, "An effective on-chip preloading scheme to reduce data access penalty," in *Proc. Supercomputing*, 1991.

[5] S. Srinath, O. Mutlu, H. Kim and Y. N. Patt, "Feedback directed prefetching: Improving the performance and bandwidth-efficiency of hardware prefetchers," in *Proc. HPCA*, 2007.

[6] M. K. Qureshi, A. Jaleel, Y. N. Patt, S. C. Steely and J. Emer, "Adaptive insertion policies for high performance caching," in *Proc. ISCA*, 2007.

[7] S. Williams, A. Waterman and D. Patterson, "Roofline: An insightful visual performance model for multicore architectures," *Communications of the ACM*, 2009.

[8] Icarus Verilog documentation, https://steveicarus.github.io/iverilog/

[9] Verilator documentation, https://verilator.org/guide/latest/

[10] cocotb documentation, https://docs.cocotb.org/

[11] GTKWave documentation, https://gtkwave.sourceforge.net/

[12] Yosys Open SYnthesis Suite, https://yosyshq.net/yosys/

[13] Project repository (this work), https://github.com/Kishore-Kotapati/micron_cache_prefetch

References [1]-[7] are background theory; refs [3]-[6] describe the classic ideas (stream buffers, stride reference prediction, feedback-directed throttling, adaptive insertion) that this design adapts. All measured results come from this project's own simulations.

Appendix A - Project File Inventory

Table 28: Source inventory

Item	Content	Used in
rtl/	8 Verilog files (7 design + DRAM model)	Ch. 3, 7
testbench/tb_system.v	trace-driven self-checking testbench	Ch. 7, 9
testbench/cocotb/test_cache.py	7 Cocotb tests	Ch. 9
python/generate_traces.py, analyze_traces.py	Level 1	Ch. 5
python/ref_cache_model.py	golden LRU model	Ch. 9
python/run_all.py	54-run regression and metrics	Ch. 9, 11
python/verilator_crosscheck.py, run_cocotb.py	second simulator, Cocotb runner	Ch. 9
python/latency_sweep.py, design_sweep.py	sensitivity and design-space studies	Ch. 11
python/evaluate_results.py, visualize_results.py	tables and plots	Ch. 11
traces/	9 workloads + demo	Ch. 5
simulation/logs/	54 Icarus logs + 54 Verilator logs	Ch. 9
simulation/waveforms/	demo_baseline.vcd, demo_final.vcd, demo.gtkw	Ch. 10
results/	CSV summaries, comparison, lint, cross-check, Cocotb, sweeps, plots	Ch. 8-11
synth/	Yosys scripts, wrapper, block diagrams, statistics, netlist	Ch. 8
docs/	architecture, algorithm, final report, learning guide, this report	-
Official slides (4 photographs)	challenge specification	Ch. 1
GTKWave screenshots (2, second machine)	waveform evidence	Ch. 10
run_all.sh, README.md	one-command reproduction and instructions	Ch. 8

Inaccessible material: none of the project's own files were missing. Conversations outside this working session were not available and are not represented.

Appendix B - Full Results Table (all architectures)

Table 29: All 54 simulations (results/performance_summary.csv)

Trace	Config	Hit %	AMAT	Cycles	Speedup	DRAM	PF iss.	Acc.+late %	Deg.
sequential	baseline	75.00	11.25	112571	1.000	2048	0	0.0	1
sequential	nextline	76.38	4.80	59723	1.885	2048	2040	100.0	1
sequential	stride	79.85	2.51	41009	2.745	2048	2039	100.0	2
sequential	stride_adapt	99.48	1.07	29214	3.853	2048	2039	100.0	7
sequential	smart_noins	99.43	1.07	29172	3.859	2048	2040	100.0	7
sequential	smart	99.43	1.07	29172	3.859	2048	2040	100.0	7
stride	baseline	0.00	42.00	133557	1.000	3000	0	0.0	1
stride	nextline	0.00	42.00	133557	1.000	5988	2988	0.0	1
stride	stride	7.60	14.05	49700	2.687	3002	2958	99.9	2
stride	stride_adapt	20.87	6.05	25705	5.196	3007	2968	99.8	7

Trace	Config	Hit %	AMAT	Cycles	Speedup	DRAM	PF iss.	Acc.+late %	Deg.
stride	smart_noins	21.10	6.04	25688	5.199	3011	2972	99.6	7
stride	smart	21.10	6.04	25688	5.199	3011	2972	99.6	7
random	baseline	0.28	41.89	177688	1.000	3989	0	0.0	1
random	nextline	0.33	41.87	177608	1.000	7962	3975	0.2	1
random	stride	0.28	41.89	177688	1.000	3989	0	0.0	2
random	stride_adapt	0.28	41.89	177688	1.000	3989	0	0.0	2
random	smart_noins	0.28	41.89	177688	1.000	4021	32	0.0	1
random	smart	0.28	41.89	177688	1.000	4021	32	0.0	1
repeated	baseline	97.50	2.02	17357	1.000	96	0	0.0	1
repeated	nextline	98.78	1.49	15318	1.133	97	51	98.0	1
repeated	stride	97.50	2.02	17357	1.000	96	0	0.0	2
repeated	stride_adapt	97.50	2.02	17357	1.000	96	0	0.0	2
repeated	smart_noins	98.28	1.71	16127	1.076	98	32	93.8	1
repeated	smart	98.28	1.71	16127	1.076	98	32	93.8	1
mixed	baseline	34.55	27.83	182100	1.000	3927	0	0.0	1
mixed	nextline	45.52	23.32	155026	1.175	7159	3922	17.7	1
mixed	stride	68.25	12.81	91942	1.981	3931	2423	99.8	2
mixed	stride_adapt	74.67	11.28	82747	2.201	3940	2454	99.5	7
mixed	smart_noins	74.25	11.34	83135	2.190	5295	3816	64.2	7
mixed	smart	72.80	11.89	86455	2.106	5652	4108	58.0	7
capacity	baseline	75.00	11.25	84464	1.000	1536	0	0.0	1
capacity	nextline	76.50	4.79	44789	1.886	1536	1530	100.0	1
capacity	stride	79.25	2.52	30809	2.742	1536	1527	100.0	2
capacity	stride_adapt	99.10	1.10	22100	3.822	1536	1527	100.0	7
capacity	smart_noins	99.14	1.09	22016	3.836	1536	1530	100.0	7
capacity	smart	99.14	1.09	22016	3.836	1536	1530	100.0	7
cnn	baseline	80.16	9.13	112549	1.000	1698	0	0.0	1
cnn	nextline	86.58	3.59	65079	1.729	1858	1675	90.7	1
cnn	stride	88.38	3.06	60541	1.859	1928	1624	86.0	2
cnn	stride_adapt	95.34	2.57	56358	1.997	2454	2155	65.2	7
cnn	smart_noins	95.75	2.02	51680	2.178	2465	2299	67.2	7
cnn	smart	95.75	2.02	51680	2.178	2465	2299	67.2	7
llm	baseline	84.38	7.41	82323	1.000	1300	0	0.0	1
llm	nextline	86.92	2.75	43557	1.890	1305	1295	99.6	1
llm	stride	93.34	1.64	34353	2.396	1312	1298	99.1	2
llm	stride_adapt	99.47	1.12	30034	2.741	1337	1323	97.2	7
llm	smart_noins	99.51	1.10	29855	2.757	1339	1330	97.1	7
llm	smart	99.51	1.10	29855	2.757	1339	1330	97.1	7
recsys	baseline	88.33	5.78	79479	1.000	1120	0	0.0	1
recsys	nextline	92.17	3.23	55005	1.445	1419	1116	75.9	1
recsys	stride	93.35	3.68	59326	1.340	1848	1224	47.1	2
recsys	stride_adapt	93.35	3.68	59326	1.340	1848	1224	47.1	2
recsys	smart_noins	93.35	3.56	58120	1.367	2202	1625	40.8	3
recsys	smart	93.85	3.17	54379	1.462	2001	1549	46.6	2

Appendix C - Verification Checklist

Table 30: Final acceptance checklist

#	Criterion	Status
1	Official specification understood (4 slides)	Complete
2	Environment checked; tools installed and versioned	Complete
3	Python trace generator working	Complete
4	Sequential, stride, random, repeated, mixed, capacity, CNN, LLM, RecSys workloads tested	Complete
5	Baseline cache implemented and matched to golden model	Complete
6	Stride predictor and prefetch engine implemented and integrated	Complete
7	Demand priority, duplicate filtering, page protection verified	Complete
8	RTL compiles without errors; Verilator lint 0 warnings	Complete
9	54/54 simulations pass; 54/54 identical on Verilator; 7/7 Cocotb	Complete
10	GTKWave waveforms generated and analysed	Complete
11	Baseline vs prefetch compared on identical traces; metrics from real logs	Complete
12	Innovation implemented and measured	Complete
13	Documentation, README with exact commands, one-command reproduction	Complete
14	Work pushed to GitHub: https://github.com/Kishore-Kotapati/micron_cache_prefetch	Complete